CONVERTING BETWEEN NUMERIC AND STRING TYPES

ABIJITH V
2024239008
COMPUTER SCIENCE

Getting ready

All the utility functions mentioned in this recipe are available in the header.

How to do it ...

- To convert from an integer or floating-point type to a string type, use std::to_string() or std::to_wstring(), as shown in the following code snippet:

```
>>auto si = std::to_string(42);
>>auto sd = std::to_wstring(42.0);
```

- To convert from a string type to an integer type, use std::stoi(), std::stol(), std::stoll(), std::stoul(), or std::stoull(), as shown in the following code snippet:

```
>>auto i1 = std::stoi("42");
```

- To convert from a string type to a floating-point type, use std::stof(), std::stod(), or std::stold(), as shown in the following code snippet:

```
>>auto d1 = std::stod("123.45");
```

There is an entire set of functions that have a name with the format ston (string to number), where n stands for i (integer), l (long), ll (long long), ul (unsigned long), or null (unsigned long long). The following list shows all these functions, each of them with two overloads—one that takes an std::string and one that takes an std::wstring as the first parameter.

```
>>int stoi(const std::string& str, std::size_t* pos = 0, int base = 10);
>>float stof(const std::string& str, std::size_t* pos = 0);
```

The way the string to integral type functions work is by discarding all white spaces before a non-whitespace character, then taking as many characters as possible to form a signed or unsigned number.

```
>>auto i1 = std::stoi("42"); // i1 = 42
>>auto i2 = std::stoi(" 42"); // i2 = 42
>>auto i3 = std::stoi(" 42fortytwo"); // i3 = 42
>>auto i4 = std::stoi("+42"); // i4 = 42
>>auto i5 = std::stoi("-42"); // i5 = -42
```

A valid integral number may consist of the following parts:

- A sign, plus (+) or minus (-) (optional)
- Prefix 0 to indicate an octal base (optional)
- Prefix 0x or 0X to indicate a hexadecimal base (optional)
- A sequence of digits

THANK YOU